
Laborationsuppgift 5: Introduktion till Fouriertransform och Diskret Fouriertransform (DFT)

Namn 1: Maximilian Andersen

Namn 2: Elias Brännström

Genom att skicka in labbrapporten intygar du/ni att följande regler har följts:

1. Laborationsuppgifter skall lösas självständigt av varje laborationsgrupp. Det är tillåtet att diskutera lösningar, men INTE att kopiera lösningar! Det är alltså INTE tillåtet att ge laborationsresultat eller färdiga lösningar till en annan grupp.
2. Alla gruppmedlemmarna förväntas ta aktiv del i genomförandet av laborationen och skrivandet av rapporten. Detta inkluderar att bygga, programmera, dokumentera, testa och felsöka. Bägge gruppmedlemmarna skall kunna svara på frågor om hur laborationen genomförts och vilka resultat som erhållits.
3. Examination baseras alltid på individuella resultat
4. Sista tillfälle för att redovisa laborationen är i augusti samma år som kursen ges.

Chahed Alasal, juni 2024

Rabaa Khatib, juni 2024

Mats Syde, januari 2025

Introduktion:

I denna laboration får du en introduktion till ett viktigt område inom signalbehandling: Fouriertransform (FT) och diskret Fouriertransform (DFT). Målet med labben är att:

- Lära dig grunderna i Fouriertransform samt deras användningsområden.
- Använda MATLAB för att beräkna Fouriertransform och analysera signaler, samt förstå konceptet med inversen av Fouriertransform.

Själva laboration är utformat för MATLAB och kodexemplen är specifikt anpassade för MATLAB. Du kan välja att använda ett annat programmeringsspråk, men vi kan inte garantera att vi kan ge hjälp med bibliotek för andra språk.

Syntaxen för MATLAB-funktioner som används för att utföra labbuppgifterna kan hittas i MATLAB-genomgången på Canvas eller genom att använda help-kommandot i MATLAB. Till exempel, om du behöver använda poly-kommandot, skriv 'help poly' i kommandofönstret för en kort beskrivning av hur det används.

Labbrapport

Efter varje avsnitt finns specifika frågor markerade i en röd ruta. Svara på dessa frågor i ett separat dokument. Inkludera all text, de begärda plottarna och koden för varje del i detta dokument, som du sedan lämnar in som din labbrapport.

Förberedelser

Innan du påbörjar labbuppgifterna, se till att besvara de följande förberedande frågorna. Detta kommer att fördjupa din förståelse av ämnet och underlätta arbetet med labbmomenten.

Moment 1 Analys av frekvensspektrum

Vi har denna signal:

$$y(x) = \sin(50.0(2.0\pi x)) + 0.5\sin(80.0(2.0\pi x))$$

Vi samplar $N = 600$ punkter av den med samplingsfrekvensen $f = 800$ Hz. Längden på signalen, L , beror på antalet sampelpunkter N och tiden mellan varje sampelpunkt T . Längden L beräknas som produkten av antalet sampelpunkter och tiden mellan dem, alltså $L = N \times T$. Där T representerar tiden mellan varje sampelpunkt och N är antalet sampelpunkter som tas under en given tidsperiod.

För att få en korrekt bild av signalens spektrum behöver du göra följande steg:

Ta den första halvan av signalen:

$$R = \text{fft}(y);$$

$$R = R(1:N/2+1);$$

Normalisera magnituden genom att dela varje värde med N :

$$R = R/N;$$

Kompensera för den borttagna halvan av datan:

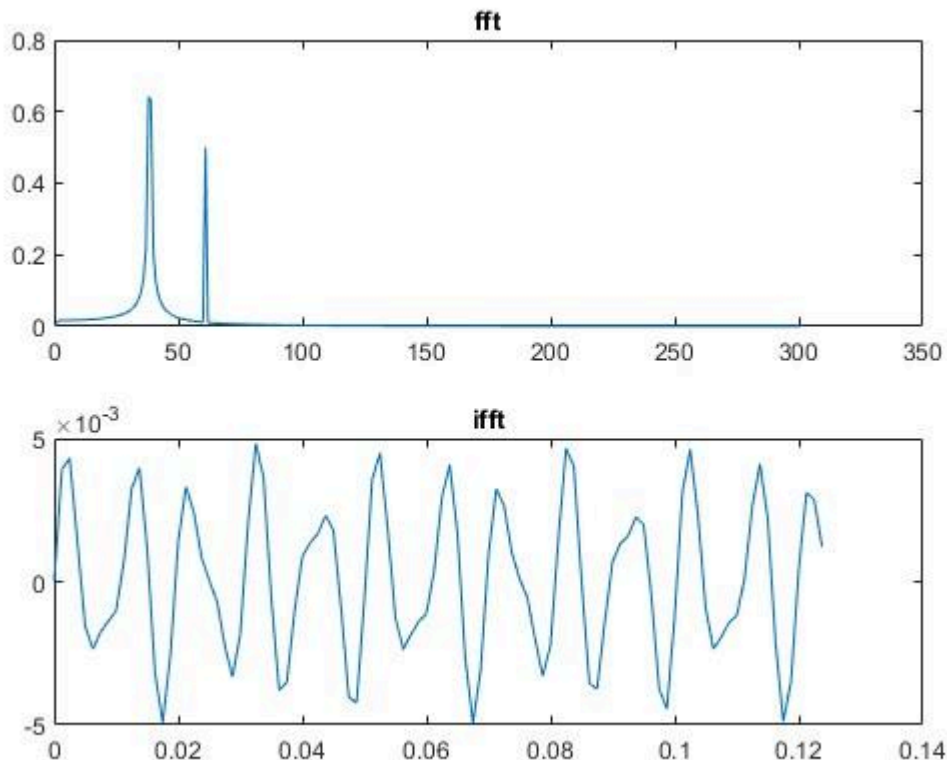
$$R(2:\text{end}-1) = 2 * R(2:\text{end}-1);$$

På så sätt ligger magnituden av FFT-utdata inom intervallet $[0:1]$.

Observera: För att plotta en figur åt gången, högerklicka på raden med plot-instruktionen och välj "Evaluate Selection in Command Window"

1. Beräkna Fouriertransformen av den givna signalen med hjälp av MATLAB:s inbyggda funktion `fft()`. Plotta utsignalen av `fft()` och inkludera resultatet i rapporten.
2. Analysera resultatet av frekvensspektrumet:
3. Hur många spikar ser du i plottningen?
4. Vilka frekvenser har spikarna?
5. Vilka amplituder har spikarna?
6. Överensstämmer de identifierade frekvenserna och amplituderna med förväntningarna baserat på signalens sammansättning?
7. Omvandla utsignalen till tidsdomän igen med hjälp av invers Fouriertransform `ifft()` och inkludera plottningen i rapporten.
8. Inkludera MATLAB-koden för de utförda uppgifterna i rapporten.

1. Beräkna Fouriertransformen av den givna signalen med hjälp av MATLAB:s inbyggda funktion `fft()`. Plotta utsignalen av `fft()` och inkludera resultatet i rapporten.



2. Analysera resultatet av frekvensspektrumet:

3. Hur många spikar ser du i plotningen?

2

4. Vilka frekvenser har spikarna?

50Hz and 80Hz

5. Vilka amplituder har spikarna?

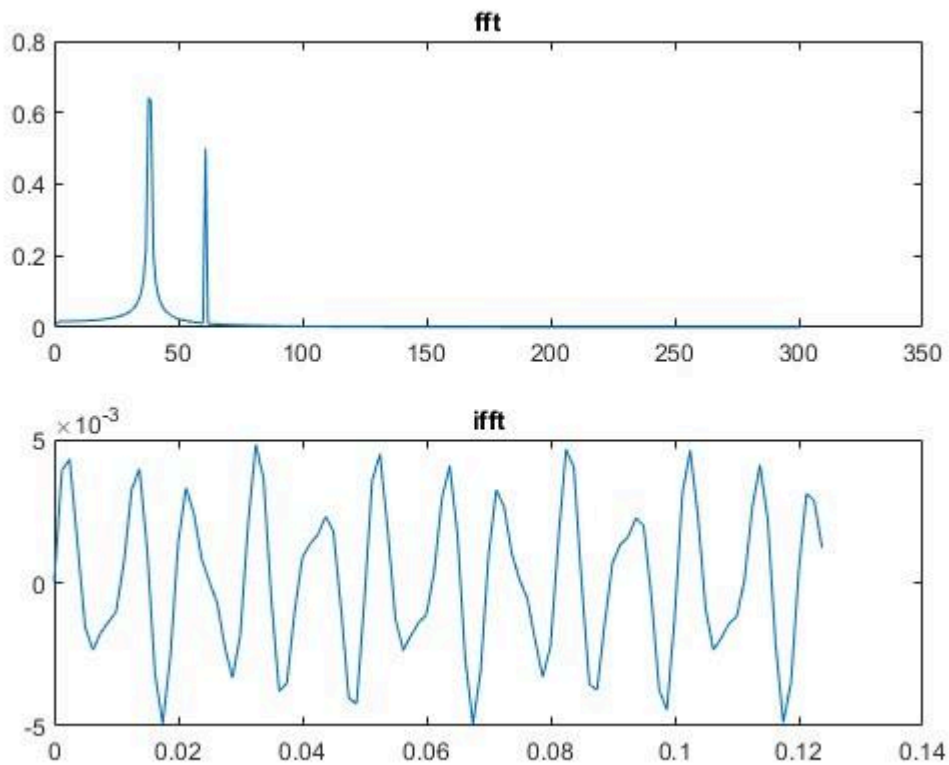
Peak 1: 0.64 and Peak 2: 0.5

6. Överensstämmer de identifierade frekvenserna och amplituderna med förväntningarna baserat på signalens sammansättning?

The identified frequencies are correct if you change what values you plot on the x-axis. If you just plot(`abs(R)`) you are plotting the indices of R on the x-axis and the value on the

y-axis. Thus you need to scale your x-axis by $\Delta f = \frac{f_s}{N}$. The amplitude for the second spike is correct but for the spike at 50Hz the amplitude is not correct. This happens because the our sampling rate is 800Hz and we have 600 samples. Giving us a sample window of $600/800 = 0.75s$. For the FFT function to calculate a perfect amplitude the sine wave needs to complete N numbers of whole cycles. $80 \cdot 0.75 = 60 \text{ cycles}$ giving us perfect amplitude. But for 50Hz $\rightarrow 50 \cdot 0.75 = 37.5$ this means the fft is spreading the amplitude over 2 indices.

7. Omvandla utsignalen till tidsdomän igen med hjälp av invers Fouriertransform `ifft()` och inkludera plotningen i rapporten.



8. Inkludera MATLAB-koden för de utförda uppgifterna i rapporten.

```
N = 600;
fs = 800;
ts = 1/fs;
L = N * ts;
x = 0:ts:(N-1)*ts;
y = sin(50*2 * pi * x) + 0.5*sin(80*2*pi*x);
R = fft(y);
R = R(1:N/2+1);
R = R/N;
R(2:end-1) = 2*R(2:end-1);
f = fs * (0:(N/2))/N;
subplot(2, 1, 1);
plot(f, abs(R));
title('fft');
tid = ifft(R);
subplot(2, 1, 2);
plot(x(1:100), tid(1:100));
title('ifft');
```

Moment 2 Implementering av DFT

Diskret Fouriertransform (DFT) beräknar ett komplext tal för varje frekvenskomponent i en diskret signal. I praktiska tillämpningar registreras data vanligtvis under en begränsad tidsram, från den första sampeln vid tidpunkt 0 till den sista sampeln vid tidpunkt $N-1$. För att beräkna den diskreta Fouriertransformen (DFT) av en diskret signal med N sampel används följande ekvation:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{\frac{-2\pi jkn}{N}} \quad (1)$$

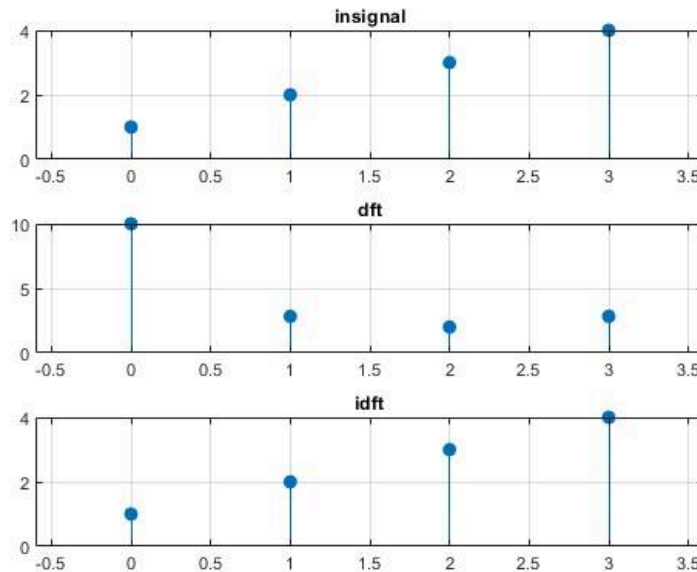
Där:

- $X[k]$ är den resulterande DFT:n, som är ett komplext tal som innehåller amplitud- och fasrelaterad information.
- $x[n]$ är amplituden av signalen vid tidpunkten n , k är frekvensindex som sträcker sig från 0 Hz till samplingsfrekvensen.
- N är antalet sampel i den diskreta signalen.

Använd följande matris av diskreta indata för denna uppgift:

$$x = [1, 2, 3, 4]$$

1. Plotta den ursprungliga signalen x med *stem()*-funktionen.
2. Skriv en MATLAB-funktion för att beräkna DFT:n av den givna signalen x med hjälp av ekvation (1). Använd sedan *stem()* för att plotta det resulterande frekvensspektrumet.
3. Beräkna och plotta inversen av DFT:n med *stem()*.
4. Analysera och jämför frekvensspektrumet för den ursprungliga signalen och dess DFT. Identifiera eventuella frekvenskomponenter och diskutera deras relativa magnituder.
5. Inkludera MATLAB-koden för de utförda uppgifterna i rapporten.



1. x
2. x
3. x
4. We can identify 4 different frequency components. Where the first spike is the DC offset and it is the sum of all the values in the time domain.
The second spike is fundamental frequency. This is the lowest non zero frequency the dft can measure for this window size.
The third frequency is the Nyquist frequency. It is the highest frequency that can be represented with a 4 point sample.
The fourth frequency is the mirrored frequency of the second one.
- 5.

```
x = [1, 2, 3, 4];
N = length(x);
n = 0:N-1;
subplot(3, 1, 1);
stem(n,x, 'filled');
title('insignal');
grid on;

X = c_dft(x);
k = 0:N-1;
subplot(3,1,2);
stem(k, abs(X), 'filled');
title('dft');
grid on;

x_inv = c_idft(X);
k = 0:N-1;
```

```
subplot(3,1,3);
stem(k, x_inv, 'filled');
title('idft');
grid on;

function X = c_dft(x)
    N = length(x);
    X = zeros(1, N);
    for k = 0:N-1
        for n = 0:N-1
            X(k+1) = X(k+1) + x(n+1) * exp(-1j * 2 * pi * k * n / N);
        end
    end
end

function x_inv = c_idft(X)
    N = length(X);
    x_inv = zeros(1, N);
    for k = 0:N-1
        for n = 0:N-1
            x_inv(k+1) = x_inv(k+1) + (1/N)*(X(n+1) * exp(1j * 2 * pi * k
* n / N));
        end
    end
end
```


Moment 3 Analysera EKG-ljudsignalen med DFT

I detta moment kommer du att tillämpa den diskreta Fouriertransformen (DFT) på en verklig ljudsignal, nämligen `ekg_without_noise.mp3`, och analysera dess betydelse inom signalbehandling.

Ladda ner filen 'ekg.mp3' från Canvas. Navigera till MATLAB:s aktuella mapp där du har sparat filen. Du kan läsa in den med kommandot:

```
[y_orig, Fs] = audioread('ekg.mp3');
```

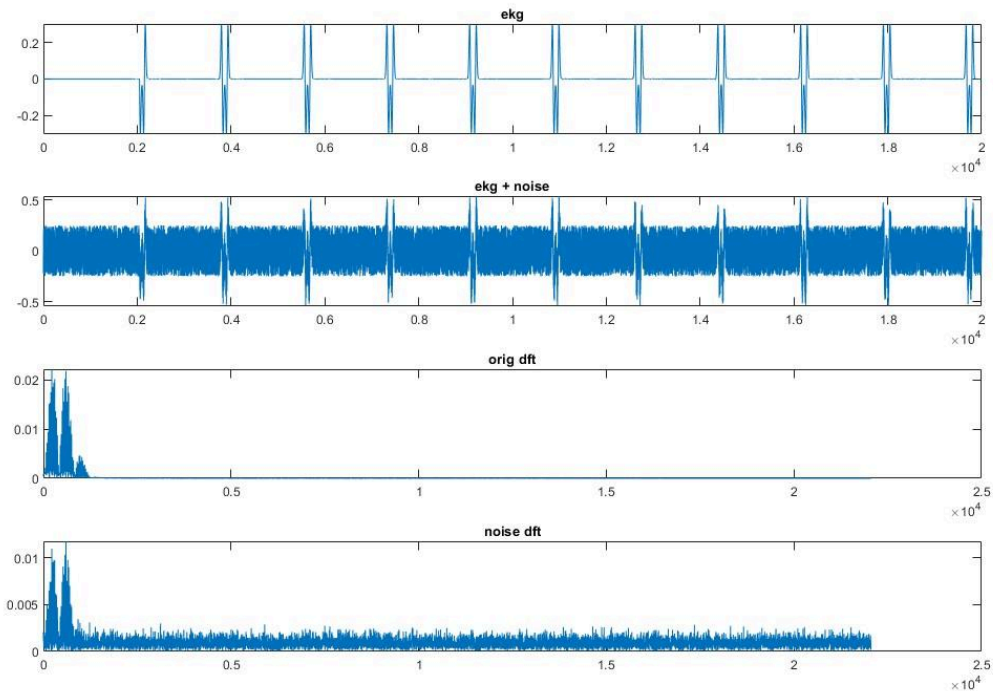
Det är en lång fil; du kan klippa 20 000 sampel från den med:

```
y_orig = y_orig(1:20000);
```

Studera hur signalen ser ut och hur dess frekvensspektrum ser ut. Låt oss lägga till lite slumpmässigt brus i den:

```
y = y_orig + rand([1:20000]) * 0.5 - 0.25;
```

1. Plotta den brusiga och den ursprungliga signalen.
2. Beräkna DFT:n av signalen från `ekg.mp3` både med och utan brus med hjälp av ekvation 1, och plotta de resulterande frekvensspektrumen.
3. Jämför den brusiga och den ursprungliga signalen samt deras frekvensspektra. Varför är brusnivån så hög i signalen men låg i frekvensspektrumet?
4. Vad är anledningen till att klippa de första 20 000 sampel från den ursprungliga ljudsignalen?
5. Inkludera MATLAB-koden i rapporten.



3. The sound of output signal with added noise is drastically different from the input file. It is much noisier. The noise is generated by adding random values to the input signal. The frequency of the noise will therefore be random as well. This means that the frequency of the noise will be equally distributed across the whole frequency spectrum, with no large frequency spikes added.

4. The first 20k samples includes the part of the signal that we want to analyze. The rest of the file is only a repetition of the signal. By only using the first 20k samples we can reduce the time required to process the signal, and because of the repetition, we don't lose any information by not using the rest of the file.

5.

```
[y_orig, Fs] = audioread('ekg.mp4');
N = 20000;
y_orig = y_orig(1:N);
subplot(4, 1, 1);
plot(f, y_orig);
title('ekg');
y_noise = y_orig + rand(1,N) * 0.5 - 0.25;
subplot(4, 1, 2);
plot(f, y_noise);
title('ekg + noise');

f = Fs * (0:(N/2))/N;

dft_y_orig = abs(c_dft(y_orig)) / N;
```

```
dft_y_orig = dft_y_orig(1:N/2+1);
dft_y_orig(2:end-1) = 2*dft_y_orig(2:end-1);
subplot(4, 1, 3);
plot(f, dft_y_orig);
title('orig dft');

dft_y_noise = abs(c_dft(y_noise))/N;
dft_y_noise = dft_y_noise(1:N/2+1);
dft_y(2:end-1) = 2*dft_y(2:end-1);
subplot(4, 1, 4);
plot(f, abs(dft_y_noise));
title('noise dft');

function X = c_dft(x)
    N = length(x);
    X = zeros(1, N);
    for k = 0:N-1
        for n = 0:N-1
            X(k+1) = X(k+1) + x(n+1) * exp(-1j * 2 * pi * k * n / N);
        end
    end
end
```

Moment 4 Ljudinspelning och brusanalys med FFT

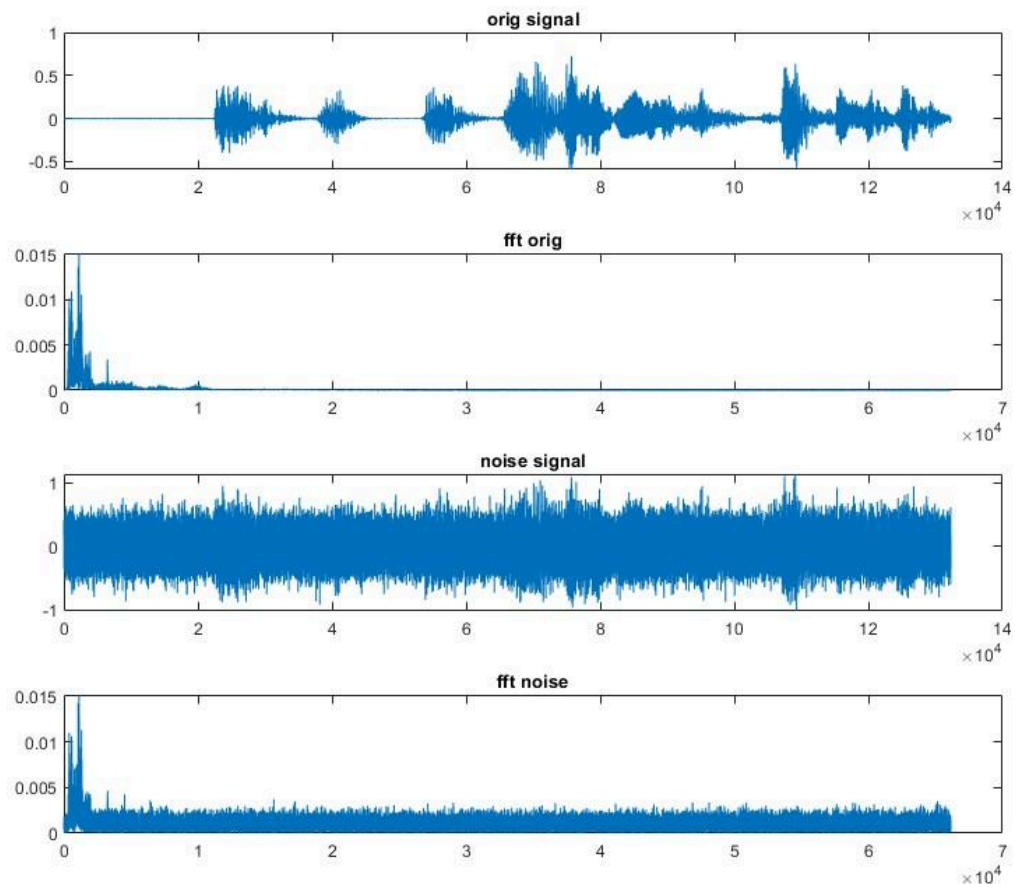
I detta moment kommer du att utföra en brusanalys med hjälp av FFT. Du kommer att börja med att spela in ljud och sedan lägga till brus i det inspelade ljudet. Därefter kommer du att använda FFT för att analysera både ljudet med och utan brus för att förstå dess frekvensinnehåll och identifiera eventuella störningar. Genom denna övning kommer du att förbättra din förståelse för FFT och dess tillämpning vid analys av ljudsignaler.

Följ dessa steg för att genomföra uppgiften:

1. Ljudinspelning: Börja med att använda funktionen *audiorecorder()* i MATLAB för att spela in ljud från mikrofonen på din dator.
2. Hämta ljuddata: Efter inspelningen, använd funktionen *getaudiodata()* för att hämta det inspelade ljudet.
3. Lägg till brus: Lägg till brus i det inspelade ljudet*.
4. Analysera ljudet med FFT: Använd MATLAB:s inbyggda funktion *fft()* för att analysera frekvensinnehållet i ljudsignalen med brus.
5. Jämför ljud med och utan brus: Utför samma analys på ljudet utan brus och jämför de två resultaten för att förstå hur brus påverkar frekvensspektra och identifiera eventuella störningar.

1. Plotta följande graf och infoga dem i rapporten: Ljudsignalen utan brus, FFT-resultatet för ljudet utan brus, Ljudsignalen med brus, FFT-resultatet för ljudet med brus.
2. Vad kan du utläsa från ljudsignalen utan brus i tidsdomänen? Vilka egenskaper eller mönster observeras i signalen?
3. Hur skiljer sig ljudsignalen med brus från den ursprungliga ljudsignalen utan brus? Finns det några tydliga förändringar i signalens amplitud eller form?
4. Vilka skillnader observerar du i amplituden och fördelningen av frekvenskomponenter mellan FFT-resultaten för ljudsignalen med och utan brus?
5. Använd *Sound*-funktionen för att lyssna på både ljudsignalen med och utan brus. Hur påverkas din upplevelse av ljudet när du jämför signalerna?
6. Hur påverkar justeringen av den valda skalfaktorn för att lägga till bruset ljudets karaktär när du antingen ökar eller minskar dess värde?
7. Inkludera MATLAB-koden i rapporten.

* Brus finns i olika varianter, var och en med sina unika egenskaper. Läs mer om dem [här](#). Du kan välja mellan att använda **randn** eller **awgn**-funktionen beroende på vilka verktyg du har tillgång till. Om du har tillgång till MATLAB Communications Toolbox, kan du använda **awgn**-funktionen för att inkludera brus med en specificerad Signal-to-Noise Ratio (SNR). För detaljer om hur du implementerar detta, klicka [här](#). Alternativt kan du använda **randn** för att generera slumpmässiga värden från en normalfördelning och sedan justera brusets intensitet genom att multiplicera med en lämplig skalfaktor



2. When no sound is present in the recording, the amplitude of the signal is at zero. When audio is present, the amplitude corresponds the sound level picked up by the microphone.

3. The noisy signal looks like the original signal, but drowned out in noise. The noise floor is at about 0.5x the original signals amplitude. The part of the signal still above the noise floor is also higher than the original signal. This is because the noise is also added to the entire recording (including the part with higher amplitude). The max amplitude should be equal to the original max + the amplitude of the noise.

4. Like the output signal in moment 3, the noise is distributed equally in the frequency domain. The high noise floor makes it difficult to see the frequencies with lower amplitude. But the higher frequency spikes are still visible, seemingly unaffected by the noise.

5. It is harder to clearly hear the voice in the signal with added noise.

6. The higher the scale factor the lower the SNR gets, which makes it harder to decipher the voice.

7.

```
duration = 3;
Fs = 44100;
scale = 0.2;
recorder = audiorecorder(Fs, 16, 1);
recordblocking(recorder, duration);
audio = recorder.getaudiodata();
%recorder.play();

N = length(audio);
t = (0:N-1)/ Fs;

noise = scale * randn(N, 1);
noise_signal = audio + noise;

fft_c = abs(fft(audio))/N;
fft_c = fft_c(1:N/2+1);
fft_c(2:end-1) = 2*fft_c(2:end-1);

subplot(4, 1, 1);
plot(audio);
title('orig signal');
subplot(4, 1, 2);
plot(fft_c);
title('fft orig');

fft_n = abs(fft(noise_signal))/N;
fft_n = fft_n(1:N/2+1);
fft_n(2:end-1) = 2*fft_n(2:end-1);

subplot(4, 1, 3);
plot(noise_signal);
title('noise signal');
subplot(4, 1, 4);
plot(fft_n);
title('fft noise');

sound(noise_signal, Fs, 16);
```

